

7.5 NON-CONTIGUOUS MEMORY ALLOCATION

In non-contiguous allocation approach, parts of a single process can occupy noncontiguous physical addresses.

7.5.1 Paging

In paging, the physical memory is divided into fixed-sized blocks called **page frames**.

Logical memory is also divided into fixed-size blocks called **pages** which are of same size as that of page frames.

When a process is to be executed, its pages can be loaded into any **unallocated frames (not necessarily contiguous) from the disk**. Figure shows two processes A and B with all their pages loaded into the memory.

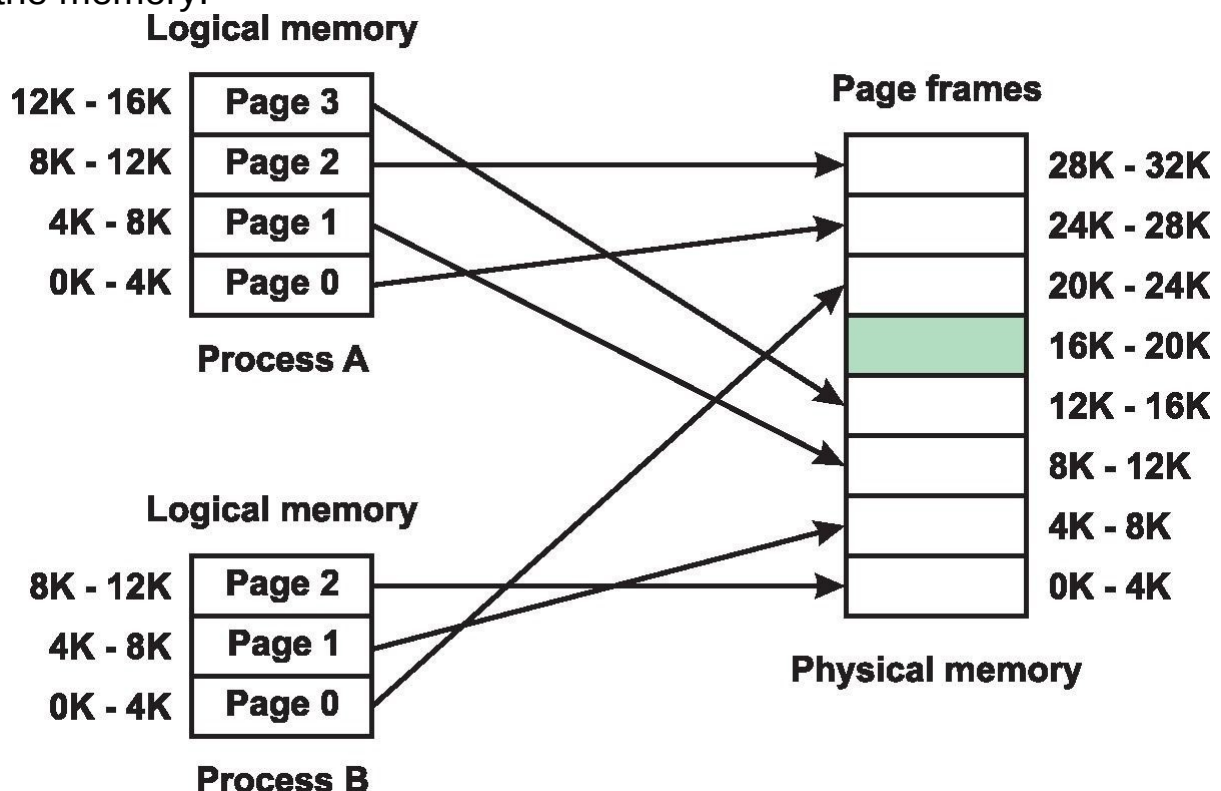


Fig. 7.11 Concept of Paging

In this figure, the page size is of 4KB. Nowadays, the systems typically support page sizes between 4KB and 8KB. However, some systems support even larger page sizes.

Basic Operation

In paging, the **mapping of logical addresses to physical addresses** is performed **at the page level**.

When CPU generates a logical address, it is divided into two parts:

- a **page number** (p) [high-order bits]
- a **page offset** (d) [low-order bits] where d specifies the address of the instruction within the page p .

Since **the logical address is a power of 2**, the **page size is always chosen as a power of 2** so that the logical address can be converted easily into page number and page offset.

Example: consider the size of **logical address space is 2^m** . Now, if we choose **a page size of 2^n** (bytes or words), then **n bits** will specify the page offset and **$m-n$ bits** will specify the page number.

How a logical address is translated into a physical address?

In paging, address translation is performed **using a mapping table**, called **page table**.

The operating system **maintains a page table for each process to keep track of which page frame is allocated to which page**.

It **stores the frame number allocated to each page and the page number is used as index to the page table** (see **Figure 7.12**).

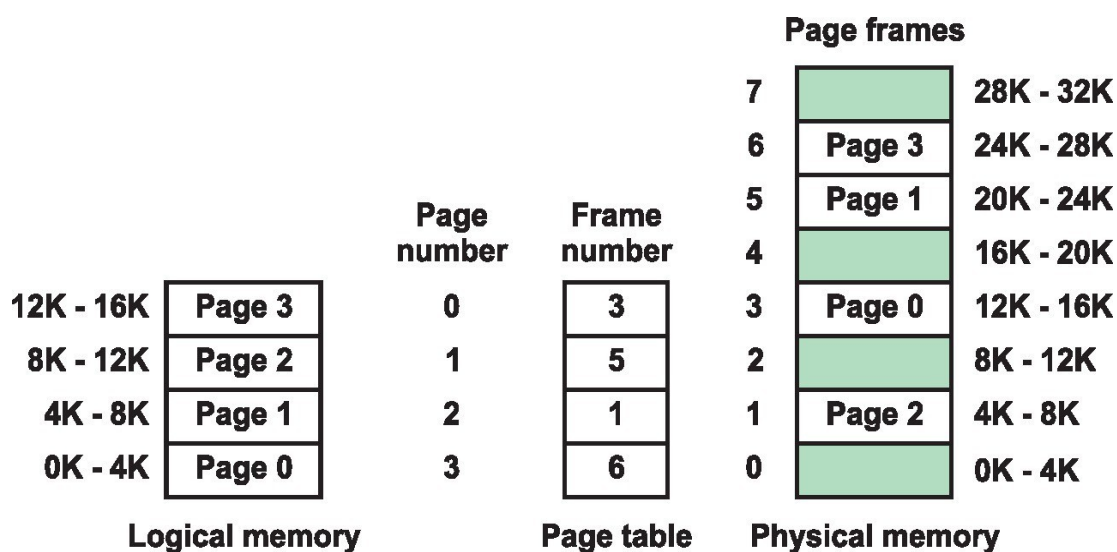


Fig. 7.12 A Page Table

When CPU generates a logical address, that address is sent to MMU.

The MMU uses the **page number** to find the corresponding **page frame number** in the page table.

That **page frame number** is attached to the high-order end of the **page offset** to form the physical address that is sent to the memory.

The mechanism of translation of logical address into physical address is shown in Figure 7.13.

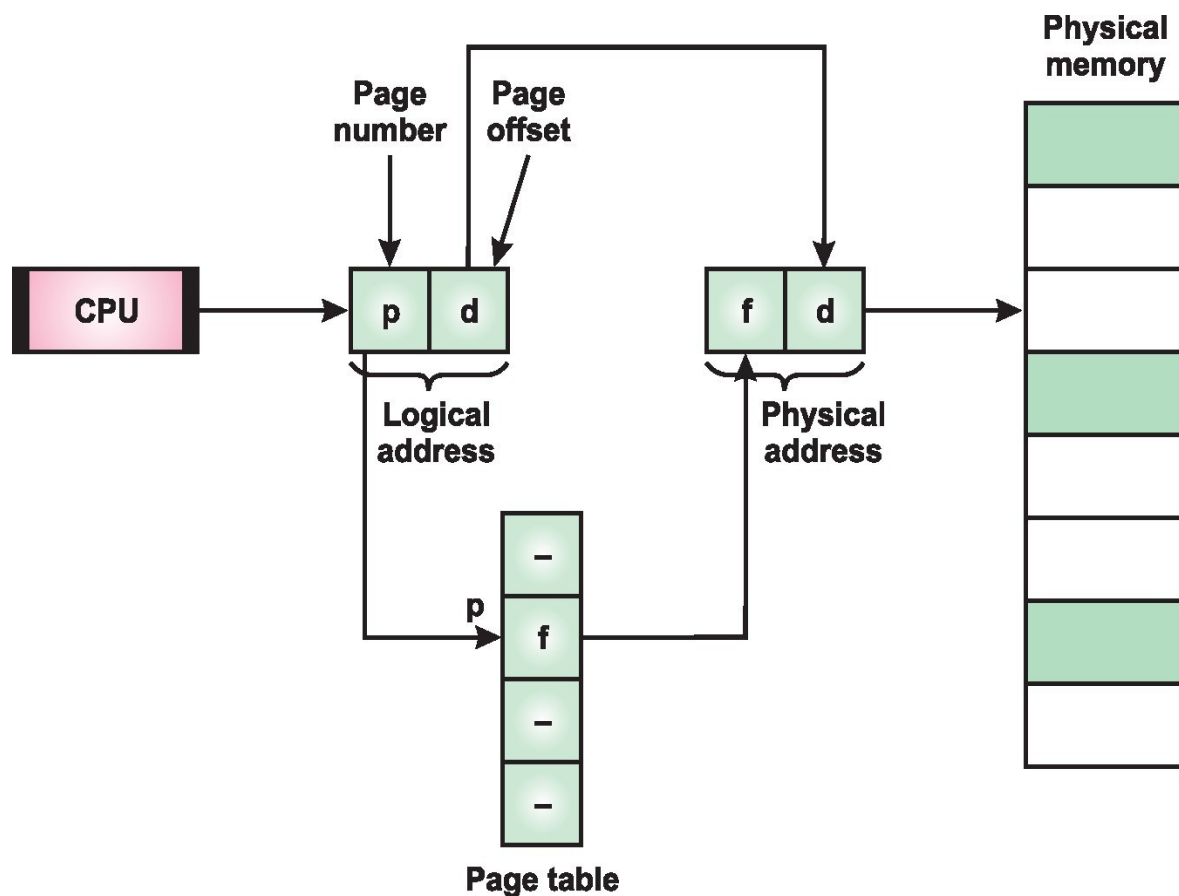


Fig. 7.13 Address Translation in Paging

Note: Since both the page and page frames are of same size, the offsets within them are identical, and need not be mapped.

Example 5 Consider a system that generates logical address of 16 bits and page size is 4KB. How many bits would specify the page number and page offset?

Solution Here, the logical address is of 16 bits, that is, the size of logical address space is 2^{16} and page size is 4KB, that is, 4×1024 bytes = 2^{12} bytes.

Thus, the page offset will be of 12 bits and page number will be of $(16-12) = 4$ bits.

Example 6 Consider a paged memory system with eight pages of 8 KB page size each and 16 page frames in memory. Using the following page table, compute the physical address for the logical address 18325.

Solution

Since, total number of pages = 8, that is, 2^3

Each page size = 8 KB, that is, 2^{13} bytes

The logical address will be of 16 bits

Out of these 16 bits, the three high-end order bits represent the page number.

13 low-end order bits represent offset within the page.

In addition, there are 16, that is, 2^4 page frames in memory, thus, the physical address will be of 17 bits.

Given logical address = 18325 which is equivalent to 0100011110010101.

In this address, page number = 010, that is, 2 and page offset = 0011110010101.

From the page table, it is clear that the page number 2 is in page frame 1011.

Therefore, the physical address = 10110011110010101, which is equivalent to 92053.

Page table

7	1010
6	0100
5	0000
4	0111
3	1101
2	1011
1	1110
0	0101

Example Consider a logical address space of eight pages of 1024 words each, mapped onto a physical memory of 32 frames.

- (i) Calculate the number of bits in the logical address.
- (ii) Calculate the number of bits in the physical address.

Solution (i)

Since total number of pages = 8, that is, 2^3

Each page size = 1024 words, that is, 2^{10} words,

The logical address will be of 13 bits (3+10).

Out of these 13 bits, the **three high-end order bits represent the page number** and the **ten low-end order bits represent offset within the page**.

(ii) There are 32, that is, 2^5 frames in the memory, thus, the physical address will be of 15 bits **(5+10)**.

Advantages

- Since the **memory** is always **allocated in fixed unit, any free frame can be allocated to a process**. Thus, there is **no external fragmentation**.

Disadvantages

- Since memory is allocated in terms of an integral number of page frames, **there may be some internal fragmentation**. That is, **if the size of a given process does not come out to be a multiple of page size, then the last frame allocated to the process may not be completely used**.

Example: consider a page size of 4KB and a process requires memory of 8195 bytes, that is, 2 pages + 3 bytes.

In this case, for only 3 bytes, an entire frame is wasted resulting in internal fragmentation.

Page Allocation

Whenever a process requests for a page frame, the operating system first locates a free page frame in the memory, and then allocates it to the requesting process.

For this, the operating system must keep track of free and allocated page frames in physical memory.

One way to achieve this is to maintain a memory-map table (MMT).

An MMT is structured in the form of a static table in which each entry describes the status of a page frame, indicating whether it is free or allocated.

Hence, an MMT for a given system contains only as many entries as there are number of page frames in the physical memory.

That is, if the size of physical memory is m and page size is p , then

$f = m/p$, where f is the number of page frames.

7	Free
6	Allocated
5	Allocated
4	Free
3	Allocated
2	Free
1	Allocated
0	Free

Fig. 7.14 Memory Map Table

Since, both m and p are usually an integer power of base 2, the resultant value of f is also an integer. A memory map table for the physical memory shown in Figure 7.12 is shown in Figure 7.14.

This approach of keeping track of free page frames is simple to implement.

However, as the number of allocated page frames increases, the system has to search more number of MMT entries to find a free page frame, which makes this approach less efficient.

Drawback: When a process **terminates**, all its page frames must be marked **FREE** in the MMT, making **deallocation time-consuming**.

Free List (Linked List)

- Free page frames are stored as a **linked list**.
- When **n free frames** are needed:
 - Remove the **first n nodes** from the free list and allocate them.
- When a process **terminates**:
 - Add its released frames to the **beginning of the free list**.
- Since the list is **unordered**, inserting at the beginning is **fast**.

Comparison

Memory Map Table (MMT)	Free List
Search time increases as memory usage increases.	Allocation and deallocation time is independent of memory utilization .
Deallocation requires updating multiple MMT entries.	Deallocation is faster by linking frames to the beginning of the list.
Uses a static table with lower memory overhead.	Uses a linked list , which has higher memory overhead but offers faster frame management .

Hardware Support for Paging

Each operating system has its own **way of storing page tables**.

1. Page Table Stored in Registers

- **Page table entries** are stored in **CPU registers**.
- **Advantages:**
 - **Very fast** access.
 - **No memory reference** is needed to access the page table.
- **Disadvantages:**

- Not suitable for **large page tables** because **registers are expensive**.
- During every **context switch**, **all registers** must be reloaded, **reducing performance**.

2. Page Table Stored in Main Memory

- The **page table** is stored in **main memory**.
- The **Page Table Base Register (PTBR)** stores the **starting address** of the page table.
- **Advantages:**
 - Only the **PTBR** needs to be reloaded during a **context switch**.
 - **Reduces context switch time**.
- **Disadvantages:**
 - Requires **two memory accesses**:
 1. Access the **page table** to find the **page frame number**.
 2. Access the **actual memory location**.
 - **Slows down memory access** (approximately **twice as many memory references**).

3. Page Table Stored in Translation Look-aside Buffer(TLB)

The system can be equipped with a special hardware device known as **Translation look-aside buffer (TLB)** (or **associative memory**).

The TLB is **inside MMU** and contains a **limited number of page table entries**.

When CPU generates a **logical address** and presents it to the **MMU**, it is **compared with the page numbers present in the TLB**.

If a **match is found** in TLB (called **TLB hit**), the corresponding page frame number is used to access the physical memory.

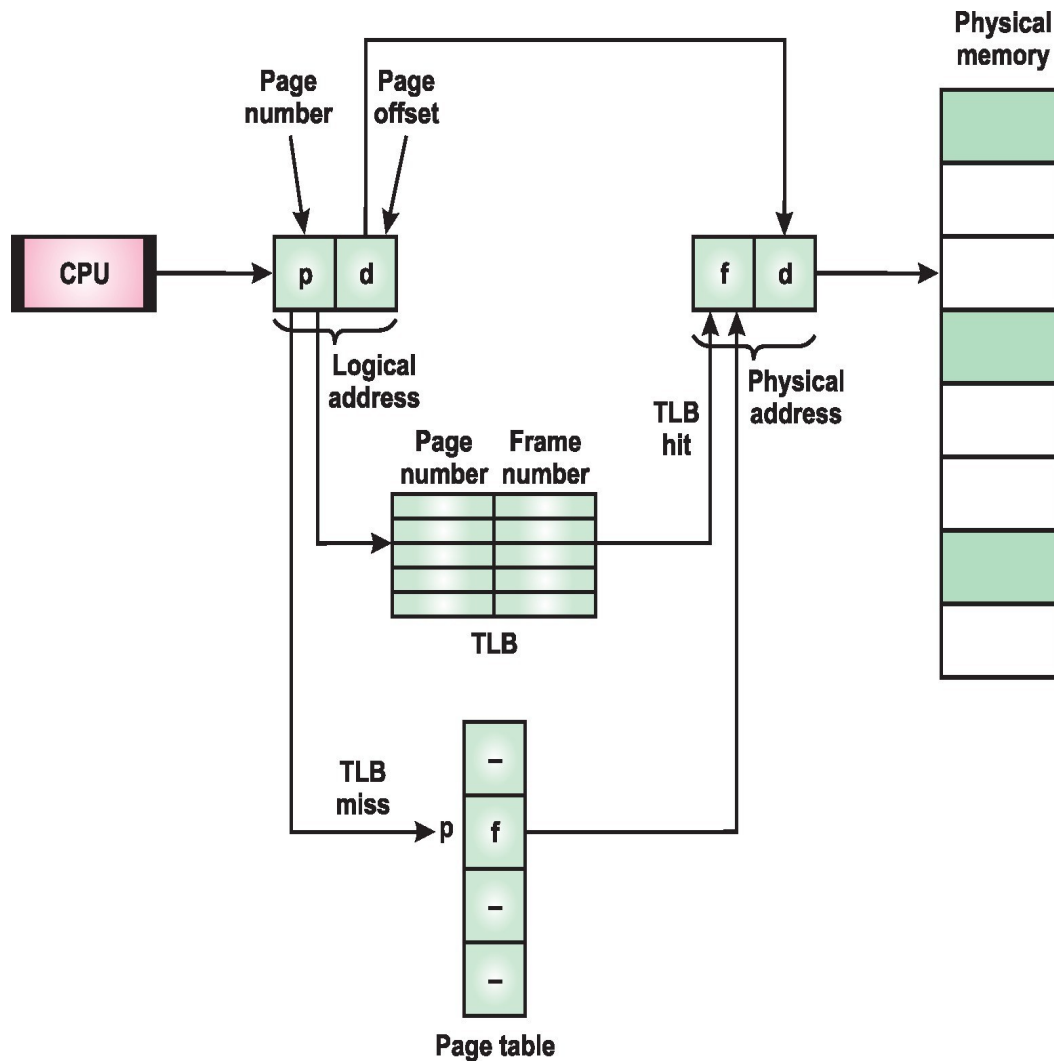
In case a **match is not found** in TLB (called **TLB miss**), **memory is referenced for the page table**.

Further, this **page number** and the corresponding **frame number** are **added to the TLB** so that **next time if this page is required, it can be**

referenced quickly.

Since the size of TLB is limited so when it is full, one entry must be replaced.

Figure 7.15 shows the mechanism of paging using TLB.



TLB can contain entries for more than one process at the same time, so there is a possibility that two processes map the same page number to different frames.

To resolve this ambiguity, a process identifier (PID) can be added with each entry of TLB.

For each memory access, the `PID` present in the TLB is matched with the value in a special register that holds the `PID` of the currently executing process.

If it matches, the page number is searched to find the page frame number; otherwise it is treated as a TLB miss.